PROGRAMMING WITH AI

From idea to a tested Python program — without prior coding experience

OUTCOME

What you should be able to do after 90 minutes

1 Translate an idea into explicit requirements

Turn “build me something” into observable behaviour and constraints.

2 Ask AI for one small implementation step

Keep changes narrow enough to understand and verify.

3 Run code and inspect what actually happens

Never equate “AI produced code” with “it works”.

4 Design tests from requirements

Define expected results before looking at actual results.

5 Debug systematically

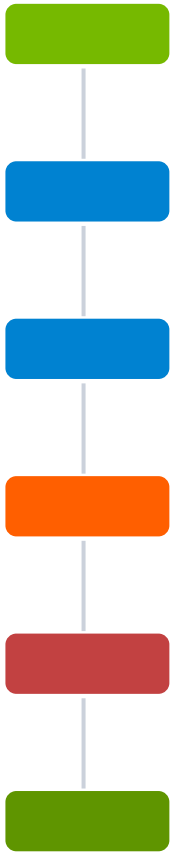
Separate diagnosis from repair; change one thing at a time.

6 Judge when you are not in control

Know when to stop, simplify, revert, or ask for explanation.

Workshop roadmap

The same development loop repeats several times; complexity increases only gradually.



FRAME VS Code + Copilot setup, control rules

SPECIFY Define requirements + examples before code

BUILD Create v0.1 → v0.4 in small increments

TEST Expected vs. actual; boundary and negative tests

DEBUG Diagnose failures, fix minimally, retest

CHALLENGE Independent extension + reflection

Rule for the whole workshop: ONE functional change → RUN → TEST → only then continue.

AI can write code. You still own the specification and the evidence.

USE AI FOR

- ✓ drafting code
- ✓ explaining unfamiliar code
- ✓ generating candidate test cases
- ✓ diagnosing error messages
- ✓ implementing one bounded change

DO NOT OUTSOURCE

- ! what the program is supposed to do
- ! whether an output is correct
- ! which cases matter
- ! whether a change broke something else
- ! whether you still understand the system

A confident explanation from an AI is not evidence that the program is correct.

Recommended workshop environment: VS Code + GitHub Copilot Free

PRIMARY PATH

VS Code + Copilot Free

1. Shows files, code, terminal and AI chat in one workspace
2. Copilot can read files in the opened project folder
3. Students learn a transferable professional development environment
4. Copilot Chat can explain, implement and diagnose bounded changes
5. Free tier is enough for this small workshop; no paid plan required

ALTERNATIVE

Google Colab

Browser-only fallback when local installation is unavailable. The development workflow stays the same.

BACKUP

Browser AI + VS Code manually

Use any permitted browser LLM to draft code, then paste changes into VS Code and run locally.

Student setup — complete this before the workshop starts

1 **Install VS Code**
Download and install Visual Studio Code from the official site.

2 **Install Python**
Install a current supported Python version. VS Code itself does not include Python.

3 **Install the Python extension**
In VS Code → Extensions, install “Python” by Microsoft.

4 **Create / sign in to GitHub**
A personal GitHub account is enough for Copilot Free. Student verification is optional for this workshop.

5 **Download the workshop folder**
Keep PROJECT_BRIEF.md, TESTS.md, PROMPTS.md, main.py and stations_workshop.csv together.

6 **Pre-work smoke test**
Open main.py, run print("Hello") and confirm that “Hello” appears in the integrated terminal.

Activate GitHub Copilot Free in VS Code

No education verification is required for the workshop: a personal GitHub account can use Copilot Free.

1 Open VS Code

Use the desktop application, not github.com.

2 Enable AI features

Click the Copilot icon and choose “Use AI Features” / “Sign up for Copilot Free”.

3 Sign in with GitHub

Complete the browser sign-in and return to VS Code.

4 Check the plan

If you have no Copilot plan, an eligible personal account is enrolled in Copilot Free.

5 Open Chat

Open Copilot Chat and keep it visible next to Explorer and the editor.

6 Test it

Ask: “Explain what a Python file is in two sentences. Do not write code.”

Workshop rule

Do not ask Copilot to edit code until the correct project folder is open. The project folder is the context boundary students should learn to control.

Copilot Student: optional upgrade

Useful for higher limits

WHAT GITHUB DOCUMENTS

Verified students can activate Copilot Student at no cost. GitHub says its education verification process is individually tailored; it does not publish a guaranteed fixed approval time.

AFTER APPROVAL

GitHub explicitly notes that the Copilot Student benefit can take several days to finish applying after education verification is approved.

APPLICATION PATH

GitHub → Settings → Education → Benefits → Start an application. After approval, activate Copilot Student separately.

Smoke test: one folder, one Python run, one Copilot read

Before the actual exercise, verify the whole toolchain with a three-minute check.

1 · OPEN THE FOLDER

```
ai-programming-workshop/
├── PROJECT_BRIEF.md
├── TESTS.md
├── PROMPTS.md
├── stations_workshop.csv
└── main.py
```

Use File → Open Folder.
Do not open just main.py.

2 · RUN PYTHON

Put this in main.py:

```
print("Hello")
```

Click ▶ Run Python File
in the top-right.

Expected terminal output:
Hello

If not: select the Python
interpreter first.

3 · TEST COPILOT CONTEXT

Open Copilot Chat:

“Read PROJECT_BRIEF.md.
Do not edit any files.
List the six requirement IDs
and explain REQ-2 in one
sentence.”

Check the answer against
the file.

WHY THIS MATTERS

This proves three separate things: Python runs locally, VS Code has the correct folder open, and Copilot can access project context only because you explicitly pointed it there.

Our development loop in VS Code

Files and tools may change; this control loop should not.



Do not ask the AI to jump from “idea” directly to “finished application”.

Project: one small CSV, one growing program

We use the same file throughout the workshop so the focus stays on requirements, tests and iteration.

```
station,line,year_opened,daily_passengers
Alexanderplatz,U5,1930,85000
Hermannplatz,U8,1927,65000
Alexanderplatz,U5,1930,85000
Wittenau,U8,1994,
Rathaus Spandau,U7,2035,41000
,U5,1973,24000
Tempelhof,U6,1929,-500
Tierpark,U5,1973,eight thousand
```

WHY ONE FILE?

1. Students upload once.
2. Each requirement maps to a visible row/case.
3. Later versions can be regression-tested on exactly the same data.
4. The dataset is small enough to inspect manually.

File supplied: stations_workshop.csv

Write the requirements before asking for code

The AI needs an explicit project brief. It cannot infer our requirements from the slide or dataset alone.

REQ-1	**Read the CSV**	The program loads stations_workshop.csv.
REQ-2	**Count rows**	Report 8 data rows; the header is not counted.
REQ-3	**Find missing values**	Report empty cells and their row/column locations.
REQ-4	**Find exact duplicates**	Report identical complete rows as duplicates.
REQ-5	**Validate year**	For non-missing values: integer from 1900 to 2026 inclusive.
REQ-6	**Validate passengers**	For non-missing values: numeric, > 0 and < 2,000,000.

The requirements are the source of truth. The code and the AI are implementations of them.

Step 1 in VS Code: give Copilot the project brief

The requirements live in PROJECT_BRIEF.md. Copilot does not know them until you explicitly provide or reference that file.

PROJECT_BRIEF.md

```
We are building a very small Python program.
I am a complete beginner. Keep the code simple and explain changes.

Requirements:
REQ-1 Read stations_workshop.csv.
REQ-2 Report the number of data rows.
REQ-3 Report missing cells with row and column.
REQ-4 Report exact duplicate rows.
REQ-5 Flag non-missing years outside 1900–2026.
REQ-6 Flag non-missing passenger values that are not numeric or <= 0.

Constraints:
- Use only Python's standard library.
- Work incrementally.
- Implement only the requirement(s) I ask for.
- Preserve behaviour that already passed its tests.
```

WHAT THE AI KNOWS

Copilot can use files in the opened VS Code workspace when you explicitly reference or attach them to the chat.

It does NOT automatically know requirements that exist only on a slide or in your head.

Therefore: point it to PROJECT_BRIEF.md before coding.

Practical rule: put requirements in a project file and explicitly tell Copilot to read that file before coding.

Where do the tests come from? From the requirements.

A test turns a requirement into a concrete example with an expected result.

REQ	BEHAVIOUR	TEST CONDITION	EXPECTED RESULT
REQ-2	Count rows	Use the supplied file	**Rows = 8**
REQ-3	Find missing values	Blank passenger + blank station	**2 missing cells**
REQ-4	Find duplicates	Alexanderplatz row appears twice	**1 duplicate row**
REQ-5	Validate year	Rathaus Spandau has 2035	**1 invalid year**
REQ-6	Validate passengers	−500 and “eight thousand”	**2 invalid non-missing values**

Formula: REQUIREMENT → example input condition → expected result. Only then run the program and compare the actual result.

Build v0.1 — REQ-1 + REQ-2 only

Open the project folder first. Then tell Copilot exactly which file contains the requirements.

```
Read PROJECT_BRIEF.md first.

Implement only REQ-1 and REQ-2 in main.py.

Use Python's standard library only.
Read stations_workshop.csv from the same folder.
Print the number of data rows.

Do not implement REQ-3 to REQ-6 yet.
Keep the code beginner-readable.
Before editing, tell me in 3 bullets what you plan to change.
```

TEST DERIVED FROM REQ-2

- ✓ Upload stations_workshop.csv
- ✓ Run the cell
- ✓ Expected: Rows = 8
- ✓ If not 8: stop and diagnose
- ✓ Only then continue to REQ-3

Run v0.1 — and understand just enough code to stay in control

You do not need to reproduce the syntax from memory. You do need a mental map.

```
import csv

with open("stations_workshop.csv",
          encoding="utf-8", newline="") as file:
    rows = list(csv.DictReader(file))

print("Rows:", len(rows))
```

import	Use Python’s built-in CSV tool — no extra package installation.
open	Open the CSV that is already in the project folder.
DictReader	Treat the first row as column names and create one record per data row.
rows	Store all data rows in a list.
len(df)	Count the data rows.
print	Show the result.

Control threshold: identify where input enters, where processing happens, and where output is produced.

Build v0.2 — add REQ-3 and immediately test it

One new behaviour; one explicit expected result.

```
The current program satisfies REQ-1 and REQ-2.

Read PROJECT_BRIEF.md again.
Implement ONLY REQ-3 in main.py.

- Report the total number of missing cells.
- List the row number and column name for each missing cell.
- Do not implement duplicate, year or passenger validation yet.
- Preserve REQ-1 and REQ-2.

Before changing code, state the expected result for our supplied CSV.
```

EXPECTED FROM REQ-3

2 missing cells:
- **Wittenau → daily_passengers**
- **blank station → station**

REGRESSION CHECK

REQ-2 must still return Rows = 8. A new feature is not acceptable if it breaks a previous requirement.

Build v0.3 and v0.4 — repeat the same engineering loop

Requirement → expected result → change → actual result → regression test.

v0.3 · REQ-4 DUPLICATES

```
Implement ONLY REQ-4.
Detect exact duplicate rows.
Report how many duplicate rows exist and show
them.
Preserve REQ-1 to REQ-3.
```

Expected from our CSV:
1 duplicate row — the second Alexanderplatz/U5/1930/85000 row.

v0.4 · REQ-5 + REQ-6 VALIDATION

```
Add REQ-5 and REQ-6.
Validate only NON-MISSING values.
Report invalid values without crashing.
Preserve REQ-1 to REQ-4.
```

Expected:
REQ-5 → 1 invalid year: 2035
REQ-6 → 2 invalid values: −500 and “eight thousand”

After every version: rerun all earlier expectations on the same CSV.

The complete acceptance test for our one dataset

Testing means comparing predetermined expectations with actual output.

REQ	MEASURE	EXPECTED	ACTUAL
REQ-2	Rows	**8**	
REQ-3	Missing cells	**2**	
REQ-4	Exact duplicates	**1**	
REQ-5	Invalid non-missing years	**1**	
REQ-6	Invalid non-missing passenger values	**2**	

If an actual value differs: do not redefine the expectation to match the program. Investigate the discrepancy.

Inspect the test data manually before trusting the code

```
station,line,year_opened,daily_passengers
Alexanderplatz,U5,1930,85000
Hermannplatz,U8,1927,65000
Alexanderplatz,U5,1930,85000   ← duplicate
Wittenau,U8,1994,              ← missing passenger
Rathaus Spandau,U7,2035,41000  ← invalid year
,U5,1973,24000                  ← missing station
Tempelhof,U6,1929,-500          ← invalid passenger
Tierpark,U5,1973,eight thousand ← invalid passenger
```

MANUAL ORACLE

Before running the program, we can count:

Rows: 8
Missing: 2
Duplicates: 1
Invalid years: 1
Invalid passengers: 2

This human-derived expectation is our test oracle.

Ask AI for additional tests — but distinguish them from the core acceptance test

The supplied CSV verifies the defined workshop requirements; extra tests probe robustness.

```
Read PROJECT_BRIEF.md and TESTS.md.

Our supplied CSV already gives one acceptance test for each
requirement.

Propose up to five ADDITIONAL edge cases that could expose bugs.
For each case state:
1. which requirement it probes,
2. the input condition,
3. the expected result,
4. why the existing dataset would not catch this bug.

Do not modify main.py yet.
```

POSSIBLE EDGE CASES

- ? year = 1900
- ? year = 2026
- ? passengers = 0
- ? passengers = 2,000,000
- ? completely empty file

Core tests come from requirements. AI-generated tests are suggestions to extend coverage.

Debugging rule: diagnose first, change code second

The same CSV already contains a controlled problematic value: “eight thousand”.

BAD DEBUGGING PROMPT

```
It does not work. Fix everything.
Make it more robust and cleaner.
```

Why this is risky:

- × scope is undefined
- × AI may change working behaviour
- × you cannot isolate what fixed the problem
- × new defects can be introduced

BETTER DEBUGGING PROMPT

```
The program fails or misclassifies the value
“eight thousand”.
Do not change the code yet.
Explain:
1. which requirement is involved,
2. where the problem occurs,
3. why this input triggers it,
4. the smallest possible fix.
```

Only after the diagnosis: implement one chosen fix and rerun REQ-2 to REQ-6.

Debugging exercise — preserve evidence

A failure is useful because it tells us which assumption was wrong.

1. Run the current program on stations_workshop.csv.
2. Record the exact error OR wrong output.
3. Identify the requirement that should govern this case.
4. Ask the AI for diagnosis only.
5. Predict the minimal change.
6. Implement that one change.
7. Rerun the complete acceptance table.

WHY RE-RUN EVERYTHING?

A fix for REQ-6 may accidentally change how missing values or row counting work.

Re-running earlier tests is called regression testing.

With one tiny dataset, regression testing is cheap.

The goal is not “no error message”. The goal is evidence that all defined requirements still hold.

When the AI changes too much: stop, compare, revert

Control is easier when each step is small enough to inspect.

1 STOP

Do not keep prompting on top of an unexplained change.

2 COMPARE

Ask: “List every behavioural change you made.”

3 REVERT

Return to the last version that passed its tests.

4 RESTATE

Repeat the requirement in narrower terms.

5 RETRY

Ask for the smallest possible modification.

6 RETEST

Run the complete acceptance table again.

“More code” is not progress. Evidence that requirements still hold is progress.

Prompts and requirements live inside the VS Code project

Students should not need to copy text from the slides. The workshop folder contains the exact text they need.

PROJECT FILES

PROJECT_BRIEF.md = requirements
TESTS.md = expected results
PROMPTS.md = copy/paste prompts
main.py = program
stations_workshop.csv = test data

Open these files directly in VS Code. This also reinforces that requirements and tests are part of the project.

WORKFLOW

Open workshop_prompts.txt.

Copy prompts from there directly into the Colab AI chat.

This avoids formatting problems and makes the exercise faster.

The revised ZIP contains exactly one CSV + one prompt text file.

Mini-challenge — choose ONE extension

A Severity levels

Classify issues as WARNING or ERROR. Define the mapping before coding.

B CSV report

Write detected issues to quality_report.csv. Define exact columns first.

C Station uniqueness

Treat station + line as a key and detect repeated keys even if other values differ.

D Configurable year

Ask the user for the latest valid year instead of hard-coding 2026.

Deliverable: one new requirement + at least two tests + working demo + one sentence on what could still fail.

Challenge workflow — 20 minutes

Do not skip straight to implementation.

1. **Write the new requirement**
   Use one sentence and make it testable.

2. **Write two expected-result tests**
   One normal case + one edge/negative case.

3. **Prompt for only this change**
   Explicitly preserve existing behaviour.

4. **Run the new tests**
   Record actual outputs, not impressions.

5. **Run regression tests**
   Run the complete acceptance table on stations_workshop.csv.

6. **Prepare a 60-second demo**
   Requirement → evidence → remaining risk.

A practical prompt pattern for AI-assisted programming

Keep this structure; adapt the details.

```
CONTEXT
Here is the current program and the requirements it already satisfies: ...

CHANGE
Implement ONLY this requirement: ...

CONSTRAINTS
Do not change ... / keep ... / use ...

ACCEPTANCE TESTS
Input ... must produce ...
Input ... must produce ...

RESPONSE
First explain your plan in 3 bullets. Then make the change. Finally list
exactly what changed and what I should retest.
```

WHY THIS WORKS

Context	reduces accidental assumptions
Change	bounds the task
Constraints	protect working behaviour
Tests	define “done”
Response	forces traceability

Five questions before you trust AI-generated code

This is the minimum control checklist for beginners.

1	**What exactly was the requirement?**	Can I state it without referring to the code?
2	**What test demonstrates it?**	Do I know the expected result before I run it?
3	**What actually happened?**	Did I record the output or only glance at it?
4	**What changed since the last working version?**	Can I name the behavioural difference?
5	**What have I not tested?**	Which input, edge case or interaction remains uncertain?

If you cannot answer these questions, simplify the program before adding features.

What you learned is transferable beyond Python

The tool may change. The engineering loop should not.

TODAY	LATER	SAME CORE
CSV checker	**web app**	**requirements**
Python / pandas	JavaScript / TypeScript	small changes
VS Code + Copilot	VS Code / other IDE / CI environment	evidence from tests
manual test matrix	automated tests	debugging discipline

AI lowers the cost of producing code. It does not lower the need for precise requirements and verification.

Current tool notes and sources — checked 18 Sep 2026

Free tiers and AI quotas can change; verify shortly before delivery.

1	**VS Code + Python**	VS Code is the editor; the Python interpreter and Microsoft Python extension are separate components. Run a .py file with “Run Python File in Terminal”.	code.visualstudio.com/docs/python/python-quick-start
2	**GitHub Copilot Free**	Copilot Free is available directly in VS Code with a monthly allowance. Verified students can activate Copilot Student at no cost.	github.com/features/copilot/plans
3	**VS Code + Copilot**	Official setup guide for using Copilot inside VS Code.	code.visualstudio.com/docs/setup/copilot
4	**Cursor Hobby**	Free Hobby tier with limited usage.	cursor.com/pricing

For a controlled teaching environment, avoid depending on any single vendor quota without a backup plan.

BUILD SMALL.
TEST EXPLICITLY.
CHANGE ONE THING.

That is the control strategy for programming with AI.

Workshop complete